

JS DOM Manipulation Methods

DOM - createElement() appendChild(), and removeChild()

1. Introduction:

- a. The `createElement()` , `appendChild()` , and `removeChild()` methods are part of the Document Object Model DOM in JavaScript.
- b. These methods are used to dynamically create, insert, and remove elements in a web page.

2. createElement():

- a. The `document.createElement()` is a method used to create the HTML element.
- b. It allows developers to dynamically create new HTML elements, such as 'span', 'p', 'img', 'table', etc.
- c. Creates a new element but does not add it to the DOM.
- d. **Syntax** →



```
let element = document.createElement("elementName");
```

e. How to Use createElement():

- i. **Create the Element:** Use the `document.createElement()` method to create a new element. This method takes one argument: the tag name of the element to be created (e.g., 'div', 'span', 'a').

- ii. **Set Attributes:** After creating the element, you can set its attributes, such as `id` , `class` , `style` , etc., using the `element.setAttribute()` method or by directly accessing the element's properties.
- iii. **Append the Element:** To display the newly created element on the page, you must append it to an existing element in the DOM using methods like `appendChild()`.

f. For Example → Creating a New Element

```
<h1>createElement Example</h1>
<button id="createButton">Create Element</button>
<div id="output"></div>
```

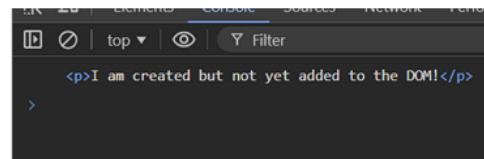
```
1 let myButton = document.getElementById("createButton").addEventListener("click", () => {
2   const newElement = document.createElement("p"); // Create a new <p> element
3   newElement.textContent = "I am created but not yet added to the DOM!";
4   document.getElementById("output").innerHTML += `<p> This paragraph is created by using createElement() dynamically.</p>`;
5   console.log(newElement);
6 });
```

- Output:

createElement Example

Create Element

This paragraph is created by using createElement() dynamically.



g. Adding Attributes:

- i. After creating an element, you might want to add attributes to it, such as `class` , or `style` .
- ii. Use the `setAttribute()` method
- iii. **Syntax** →

 `element.setAttribute(key, value);`

- iv. Key Name → `class`, `id`, etc (attribute name)

- v. Value → value of the attribute
- h. If the attribute already exists, the current value will be overridden by the new value.
- i. For Example →

```
1 let myButton = document.getElementById("createButton").addEventListener("click", () => {
2   const newElement = document.createElement("p"); // Create a new <p> element
3   newElement.textContent = "This paragraph is created by using createElement() dynamically.";
4   document.getElementById("output").innerHTML =
5     `<p> This paragraph is created by using createElement() dynamically.</p>`;
6
7   //Adding the id and class to the paragraph.
8   newElement.setAttribute('id', 'paraId');
9   newElement.setAttribute('class', 'paraClass');
10  console.log(newElement);
11 });
```

- Output:

```
<p id="paraId" class="paraClass">This paragraph is created by using createElement() dynamically.</p>
```

- j. Another Way to Add Attributes to the element directly:

```
1 let myButton = document.getElementById("createButton").addEventListener("click", () => {
2   const newElement = document.createElement("p"); // Create a new <p> element
3   newElement.textContent = "This paragraph is created by using createElement() dynamically.";
4   document.getElementById("output").innerHTML =
5     `<p> This paragraph is created by using createElement() dynamically.</p>`;
6
7   //Adding the id and class to the paragraph.
8   newElement.id = "paraId";
9   newElement.className = "paraClass";
10  console.log(newElement);
11 });
```

- Same Output:

```
<p id="paraId" class="paraClass">This paragraph is created by using createElement() dynamically.</p>
```

3. appendChild():

- a. The DOM `appendChild()` method adds a new child node to the end of a specified parent node's child list.
- b. It can move existing nodes or add new nodes, allowing dynamic updates to the document structure by appending elements, text, or other nodes.

c. Syntax →



```
element.appendChild(node);
```

Here element -> Parent

d. For Example →

```
1 document.getElementById("createButton").addEventListener("click", () => {
2   const newElement = document.createElement("p"); // Create a new <p> element
3   newElement.textContent = "I am added to the DOM using appendChild()";
4
5   //append or add to container.
6   document.getElementById("output").appendChild(newElement);
7   console.log(newElement);
8 });
```

- Output:

createElement() and appendChild() Example

Create Element

I am added to the DOM using appendChild()

4. `removeChild()`:

- a. The DOM `removeChild()` method is used to remove a specified child node of the given element.
- b. It returns the removed node as a node object or null if the node doesn't exist.

c. **Syntax** →



```
element.removeChild(node);
```

DOM - innerHTML, innerText, and textContent Property

1. Introduction:

- a. In HTML, *innerHTML* , *innerText* , and *textContent* are properties of the DOM (Document Object Model).
- b. They allow you to read and update the content of HTML elements.

2. What is the *innerHTML* Property?

- a. The *innerHTML* returns everything inside it including all HTML elements, child tags, and text content and it is used to add HTML to the element.
- b. When you use the *innerHTML* property, it reads both the HTML markup and the text content of the element.
- c. **Syntax of innerHTML (to read) Property →**



element.innerHTML

- d. **Syntax of innerHTML to update existing HTML content with new content →**



element.innerHTML = text

- e. **Return Value** ⇒ This property returns a string that contains the HTML content within an element.
- f. **For Example** →

```
<p id="para">Hello World!</p>
```

```
1 let paraElement = document.getElementById('para');  
2 console.log(paraElement.innerHTML); //only text present inside the tag  
3 console.log(paraElement.outerHTML); //return whole html element
```

- Output:

```
Hello World!  
  
<p id="para">Hello World!</p>
```

3. What is the **innerText** Property?

- The DOM **innerText** property is used to retrieve (read) or change (update) the visible text content directly within an HTML element on a webpage.
- When you use **innerText** to read the content of an element, it returns the text as it appears on screen.
- It ignores HTML tags. And it also does not include text that is hidden with CSS styles.
- Syntax of the HTML DOM innerText (to read the visible content) property ⇒**



element.innerText

- Syntax for to update the existing visible HTML content with new content ⇒**



element.innerText = text

f. For Example →

```
<p id="para">Hello World!</p>
```

```
1 let paraElement = document.getElementById("para");  
2 console.log(paraElement.innerText); //return only the text only.
```

- Output:

```
Hello World!
```

4. What is the **textContent** Property?

- The DOM **textContent** property is used to access and update the text content of an HTML element and all its child elements as a single string.
- The **textContent** property also ignores all HTML tags and returns only the text.
- It also returns all text, whether it's rendered on screen or not.
- Syntax of the DOM textContent (to access the content) property ⇒**



```
element.textContent
```

- Syntax for to update the existing HTML content with new content ⇒**



```
element.textContent = new_text
```

```
<h1 id="myElement">textContent Property Example</h1>
<button id="btn"></button>
```



```
1 let element = document.getElementById('myElement');
2 console.log(element.textContent);
```

- Output:

textContent Property Example

- Setting the value to the element ⇒



```
1 let btn = document.getElementById('btn');
2 btn.textContent = 'Click Me!'; //Setting the value to the button
3 console.log(btn.textContent);
```

- Output:

textContent Property Example

Before: After:

Click Me!

5. Final Conclusion:

- a. *innerHTML* ⇒ returns everything inside it including all HTML elements, child tags, and text content and it is used to add HTML to the element.
- b. *innerText* ⇒ returns the text that is visible on the screen means it excludes hidden text and the tags. It shows the text as it is visible on the screen. It excludes the hidden text and the HTML tags. It is used to add the text to an element.
- c. *textContent* ⇒ returns complete text including the hidden text and hidden HTML tags. It shows exactly as the code is laid out in the source code.